

# PYTHON PROGRAMMING (BCSE-004)

University-Standard Comprehensive Master Notes • Complete 4-Unit Syllabus

Course Code: **BCSE-004** Degree: **B. Tech. 2nd Semester** Curriculum: **Common to All Streams** Credits: **2.0 (L:2, T:0, P:0)**

Total Marks: **100 (Internal: 40, End Sem: 60)** Exam Standard: **M.M. (Deemed to be University)**

## UNIT I: Introduction to Programming and Python

CORE FUNDAMENTALS • 25% EXAM WEIGHTAGE

### 1.1 Fundamentals of Programming, Algorithms & Language Generations

A **computer program** is a deterministic sequence of instructions executed by a central processing unit (CPU) to perform a computational task. The design of reliable software begins with an **Algorithm**: a finite, unambiguous, step-by-step procedure that accepts specified inputs and yields a well-defined output within finite time.



Figure 1.1: The 4-Stage Software Engineering Lifecycle from Problem Statement to Machine Execution

| Classification                      | Abstraction Level                               | Execution Translation                            | Platform Portability                     | Examples                          |
|-------------------------------------|-------------------------------------------------|--------------------------------------------------|------------------------------------------|-----------------------------------|
| <b>Machine Language (1GL)</b>       | Lowest (Binary: 0s and 1s)                      | Direct native CPU microcode execution            | Zero (tied to specific CPU architecture) | Machine opcode: 10110000 01100001 |
| <b>Assembly Language (2GL)</b>      | Low (Mnemonics & registers)                     | Translated via <i>Assembler</i>                  | Zero (specific to register sets x86/ARM) | MOV AX, 01; ADD BX, AX            |
| <b>Compiled High-Level (3GL)</b>    | High (Mathematical / English-like)              | Ahead-of-Time (AOT) to standalone machine binary | Source portable; binary non-portable     | C, C++, Rust, Go                  |
| <b>Interpreted High-Level (4GL)</b> | Very High (Object-oriented & dynamically typed) | Line-by-line Bytecode interpretation in VM       | <b>100% Write Once, Run Anywhere</b>     | <b>Python</b> , JavaScript, Ruby  |

### 1.2 Origin, Evolution & Philosophy of Python

Python was conceived in late 1989 by Dutch computer scientist **Guido van Rossum** at Centrum Wiskunde & Informatica (CWI) in the Netherlands as a successor to the ABC programming language. It was officially released in February 1991. The language is named after the British comedy troupe *Monty Python's Flying Circus*.

- **Python 2.0 (October 2000)**: Introduced list comprehensions, cycle-detecting garbage collector, and Unicode support. Reached official End-of-Life (EOL) on January 1, 2020.
- **Python 3.0 (December 2008)**: A backward-incompatible release designed to rectify fundamental design flaws. Print was elevated from a keyword statement to a built-in function `print()`, strings became Unicode by default, and integer division `/` returns a float while `//` performs integer floor division.

**The Zen of Python (PEP 20)**: Core philosophical tenets authored by Tim Peters accessible via `import this`:

- *Beautiful is better than ugly.*
- *Explicit is better than implicit.*
- *Simple is better than complex.*
- *Readability counts.*
- *Special cases aren't special enough to break the rules.*
- *Errors should never pass silently.*

### 1.3 Python Internal Execution Architecture: Compiler vs PVM

Contrary to the common misconception that Python is a purely interpreted language, CPython uses a two-tier hybrid execution pipeline. Source code (.py) is first automatically compiled into an intermediate, platform-independent representation known as **Bytecode** (stored as .pyc inside the `__pycache__` directory). This bytecode is then interpreted and executed by the **Python Virtual Machine (PVM)**.

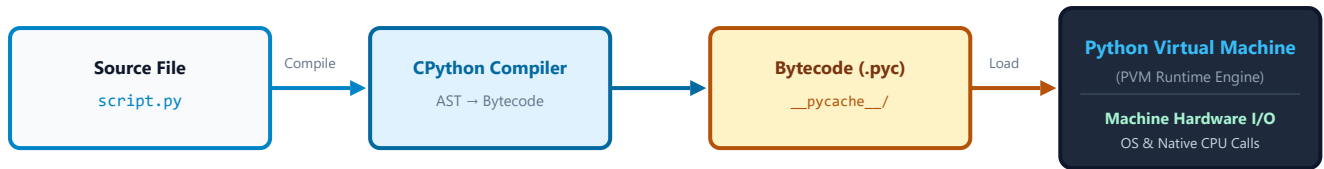


Figure 1.2: CPython Hybrid Execution Pipeline: Source Code to Bytecode to Python Virtual Machine (PVM)

## 1.4 Variables & Memory Reference Model

In statically typed languages such as C, a variable is a **named memory container** with fixed size and fixed type. In Python, variables are **symbolic references (pointers)** bound to objects in dynamic heap memory.

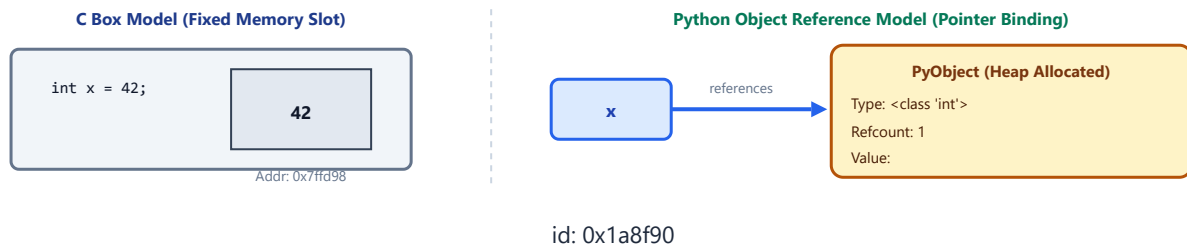


Figure 1.3: Contrasting the C Memory Storage Box vs Python's Dynamic Object Reference Model

```
# Verification of Object Identity and Reference Binding
a = 256
b = 256
print(a is b)      # True (Integer interning for range [-5, 256])
print(id(a), id(b)) # Identical memory addresses

x = [1, 2, 3]
y = x              # y references the SAME list object
y.append(4)
print(x)           # Output: [1, 2, 3, 4] <-- x reflects the modification!
>>> True | [1, 2, 3, 4]
```

## 1.5 Python Tokens: Keywords, Identifiers, Indentation & Literals

- Indentation:** Python utilizes whitespace indentation instead of curly braces { } to delimit code blocks. Standard convention (PEP 8) mandates **4 spaces per indentation level**. Mixing tabs and spaces triggers a fatal `TabError`.
- Keywords:** 35 reserved words in Python 3.10+ (accessible via `import keyword; print(keyword.kwlist)`). Keywords cannot be used as identifier names.
- Identifiers:** User-defined names for variables, functions, and classes. Rules: Must begin with an alphabet [a-zA-Z] or underscore `_`; cannot start with a digit; alphanumeric after first char; case-sensitive (`Temp` ≠ `temp`).
- Literals:** Raw data values. Types include:
  - Numeric:* Integer (42, 0b101, 0o52, 0x2A), Float (3.14159, 1.5e-4), Complex (3 + 4j).
  - String:* Single/Double quoted, Triple quoted (multi-line), Raw string (`r"C:\otes ew"` ignores escape sequences).
  - Boolean & None:* `True` (evaluates to 1), `False` (evaluates to 0), and `None` (singleton representing absence of value).

## 1.6 Exhaustive Classification of Python Operators

| Operator Category | Symbols               | Behavior & University Nuance                                                   | Code Snippet & Output                           |
|-------------------|-----------------------|--------------------------------------------------------------------------------|-------------------------------------------------|
| Arithmetic        | +, -, *, /, //, %, ** | / yields true float; // yields floor quotient; ** exponentiates right-to-left. | 7 // 2 == 3 • 7 / 2 == 3.5 • 2 ** 3 ** 2 == 512 |
| Relational        | ==, !=, <, >, <=, >=  | Evaluates operand values; supports mathematical chaining.                      | 5 < 10 < 20 evaluates to True                   |
| Logical (Boolean) | and, or, not          | Short-circuit evaluation: x and y returns x if false, else y.                  | 0 and "Hello" → 0 • [] or [1] → [1]             |

| Operator Category                | Symbols                | Behavior & University Nuance                                                   | Code Snippet & Output                        |
|----------------------------------|------------------------|--------------------------------------------------------------------------------|----------------------------------------------|
| <b>Bitwise</b>                   | &,  , ^, ~, <<, >>     | Operates on binary bit patterns; ~x equals -(x + 1) (two's complement).        | 5 & 3 == 1 • ~5 == -6 • 4 << 2 == 16         |
| <b>Assignment &amp; Walrus</b>   | =, +=, -=, *=, /=, :=  | Walrus := (Python 3.8+) assigns and returns value in an expression.            | while (n := len(lst)) > 0:<br>lst.pop()      |
| <b>Membership &amp; Identity</b> | in, not in, is, is not | in searches sequence items; is tests pointer memory identity (id(a) == id(b)). | [1] == [1] is True, but [1] is [1] is False! |

#### Why Python Does NOT Support ++ or --:

In C, x++ mutates the variable memory location in-place. In Python, integers are **immutable objects**. If Python permitted x++, it would break object immutability. Furthermore, writing ++x is parsed as two unary positive operators: +(+(x)), which simply evaluates to x without mutating it! Always write x += 1.

## 2.1 Decision Making & Conditional Branching

Control flow structures alter the sequential linear execution of code based on dynamic boolean expressions. In Python, every expression evaluates to either **truthy** or **falsey**. Falsy constants include: `False`, `None`, `0`, `0.0`, `""` (empty string), `[]` (empty list), `()` (empty tuple), and `{}` (empty dictionary). All other values are truthy.



Figure 2.1: Multi-way Branching Architecture using if-elif-else Conditional Chains

## 2.2 Looping Constructs: while, for, and the Unique Loop-else Clause

### while Loop (Indefinite Iteration)

Executes a target block repeatedly as long as a governing condition remains True.

```
count = 0
while count < 3:
    print(f"Iteration {count}")
    count += 1
else:
    print("Exited normally without break")
```

### for Loop (Definite Sequence Traversal)

Iterates directly over elements of any iterable (list, string, range, tuple).

```
# range(start, stop, step)
for i in range(1, 6, 2):
    print(i, end=" ")
else:
    print("Done")

>>> 1 3 5
Done
```

## 2.3 Comparative Analysis: break vs continue vs pass

| Feature / Metric     | break Statement                                  | continue Statement                                     | pass Statement                                    |
|----------------------|--------------------------------------------------|--------------------------------------------------------|---------------------------------------------------|
| Primary Action       | Terminates the loop immediately.                 | Skips the remainder of the current cycle.              | Does absolutely nothing (NOP).                    |
| Program Counter Flow | Jumps to the first line <i>outside</i> the loop. | Jumps directly to the loop test condition / next item. | Flows to the very next line of code sequentially. |
| Effect on Loop else  | <b>Suppresses</b> the else block from executing. | Does <b>not</b> suppress; loop else executes.          | Has zero effect on loop execution flow.           |
| Primary Use Case     | Early exit on finding a target element.          | Filtering invalid data entries during traversal.       | Syntactic placeholder for empty stubs.            |

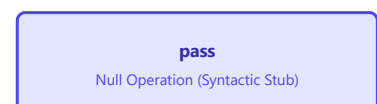
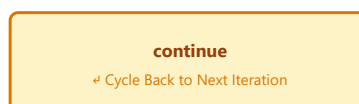
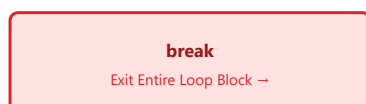


Figure 2.2: Visual Execution Path Comparison of Loop Control Primitives

## 2.4 Functions: Signatures, Invocation & Argument Mechanics

A **function** is a reusable, self-contained block of organized instructions that performs a modular sub-task. Functions provide abstraction, eliminate redundancy (DRY principle), and enable unit testing.

```
def calculate_bill(customer, *items, tax_rate=0.05, **metadata):
    '''Comprehensive demonstration of positional, *args, default, and **kwargs.'''
    subtotal = sum(items)
    total = subtotal * (1 + tax_rate)
    print(f"Customer: {customer} | Total Bill: {total:.2f} INR")
    for key, value in metadata.items():
        print(f"    {key}: {value}")
    return total

# Calling with heterogeneous argument configurations
calculate_bill("Bhavya", 120, 45, 35, tax_rate=0.10, payment="UPI", table=4)

>>> Customer: Bhavya | Total Bill: 220.00 INR
      payment: UPI
      table: 4
```

## 2.5 Variable Scope & The LEGB Name Resolution Rule

When an identifier is referenced inside a function, Python searches four concentric scope namespaces in the strict order **L** → **E** → **G** → **B**:



Figure 2.3: The LEGB Name Resolution Protocol • Python Search Order for Unqualified Names

## 2.6 Anonymous Functions & Date/Time Library Operations

- **Lambda Functions:** Single-expression inline anonymous functions defined with `lambda arguments: expression`. Commonly combined with higher-order functions:

```
nums = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, nums))      # [1, 4, 9, 16, 25]
evens = list(filter(lambda x: x % 2 == 0, nums)) # [2, 4]
```

- **Date & Time Module (datetime):** Core classes include `date`, `time`, `datetime`, and `timedelta`.

```
from datetime import datetime, timedelta
now = datetime.now()
formatted = now.strftime("%d-%b-%Y %H:%M:%S") # e.g., '19-Sep-2026 14:30:00'
parsed = datetime.strptime("25-12-2026", "%d-%m-%Y")
next_week = now + timedelta(days=7)           # Arithmetic via timedelta
```

### 3.1 Python Lists: Dynamic Arrays & Operations

A **List** is an ordered, mutable, heterogeneous sequence of object references. Internally, CPython implements lists as **contiguous dynamic arrays of 8-byte pointer addresses** with geometric over-allocation (growth sequence: 0, 4, 8, 16, 25, 35, 46, 58...) enabling amortized  $O(1)$  append operations.

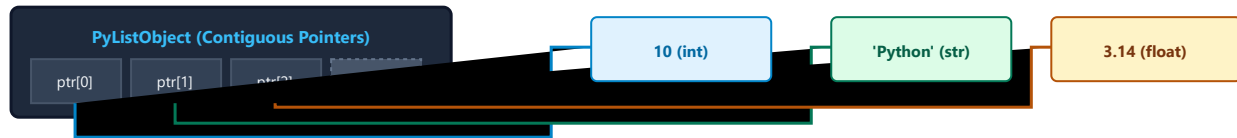


Figure 3.1: Internal Memory Layout of a Python List Showing Pointers Referencing Heterogeneous Heap Objects

```
# Fundamental List Primitives & Methods
fruits = ['apple', 'banana', 'cherry']
fruits.append('date')           # Appends to end: ['apple', 'banana', 'cherry', 'date']
fruits.insert(1, 'blueberry')   # Insert at index 1: O(n) element shift
removed = fruits.pop()          # Removes & returns last item: 'date'
fruits.remove('banana')         # Removes first match; raises ValueError if absent

# Advanced Slicing Syntax: [start : stop : step]
seq = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(seq[2:8:2])               # [2, 4, 6]
print(seq[::-1])               # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] (O(n) string/List reverse)
```

### 3.2 Tuples: Immutability, Traps & List vs Tuple Analysis

A **Tuple** is an ordered, *immutable* sequence defined by parentheses ( ) or comma separation. Once instantiated, its length and its reference elements cannot be reassigned.

**The Single-Element Tuple Comma Requirement:** Writing `t = (5)` evaluates to integer 5. To create a 1-tuple, a trailing comma is mandatory: `t = (5,)`!

| Comparison Dimension            | Python List (list)                                                           | Python Tuple (tuple)                                               |
|---------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>Mutability</b>               | <b>Mutable:</b> Can add, remove, and update items in-place.                  | <b>Immutable:</b> Elements cannot be altered once allocated.       |
| <b>Memory Overhead</b>          | Higher (pre-allocates spare buffer for dynamic growth).                      | Lower (compact allocation sized precisely to element count).       |
| <b>Hashability as Dict Keys</b> | <b>Unhashable:</b> Raises <code>TypeError: unhashable type</code> .          | <b>Hashable:</b> Can serve as dict keys if all elements immutable. |
| <b>Iteration Performance</b>    | Slightly slower due to pointer indirection and over-allocation.              | Faster (recognized by compiler optimizations).                     |
| <b>Modifying Workaround</b>     | Direct in-place methods ( <code>.append()</code> , <code>.extend()</code> ). | Must convert: <code>t = tuple(list(t) + [new_val])</code> .        |

### 3.3 Python Dictionaries: Hash Tables & Key Properties

A **Dictionary** is a mapping structure storing associative {key: value} pairs. Since Python 3.7, dictionaries maintain **insertion order** by maintaining an internal sparse hash index table referencing a dense array of entries.

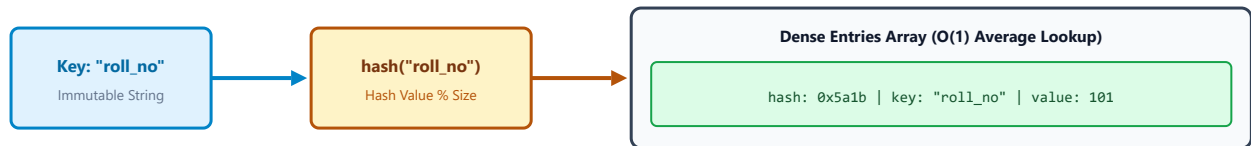


Figure 3.2: Python Dictionary Hash Table Mechanism: Key Hashing to Dense Key-Value Storage

### Properties of Dictionary Keys:

1. **Must be Hashable & Immutable:** Keys must implement `__hash__()` and `__eq__()`. Strings, integers, floats, and tuples containing only immutables are valid keys. Mutable objects (lists, dictionaries, sets) raise `TypeError: unhashable type`.
2. **Keys Must be Unique:** Duplicate keys are not permitted; declaring duplicate keys silently overwrites the previous value with the latter assignment.

```
# Dictionary Manipulation & Comprehensions
student = {'id': 101, 'name': 'Bhavya', 'cgpa': 9.8}
cgpa = student.get('cgpa', 0.0) # Safe access (avoids KeyError)
scholarship = student.setdefault('scholarship', 'Merit')

# High-Yield Dictionary Comprehension
matrix = {x: x**3 for x in range(1, 6) if x % 2 != 0}
print(matrix) # Output: {1: 1, 3: 27, 5: 125}
```

## 4.1 File Handling: Access Modes, Context Managers & File Pointers

Files provide persistent secondary storage. Python abstracts files into stream objects. Always utilize the **Context Manager** (with `open(...)` as `f:`) because it invokes the `__enter__` and `__exit__` dunder protocol, guaranteeing that the file handle is reliably closed even if runtime exceptions are raised.

| Mode                 | File Pointer Placement      | File Existence Requirement                                           | Truncation Behavior                                             |
|----------------------|-----------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------|
| 'r' (Read Only)      | At the beginning (offset 0) | <b>File must exist</b> (else raises <code>FileNotFoundError</code> ) | No truncation.                                                  |
| 'w' (Write Only)     | At the beginning (offset 0) | Created automatically if absent                                      | <b>Immediate truncation to 0 bytes!</b>                         |
| 'a' (Append Only)    | At the end (EOF)            | Created automatically if absent                                      | No truncation; writes strictly append.                          |
| 'r+' (Read & Write)  | At the beginning (offset 0) | <b>File must exist</b>                                               | Overwrites existing data in-place from pointer.                 |
| 'w+' (Write & Read)  | At the beginning (offset 0) | Created automatically if absent                                      | <b>Truncates existing file to 0 bytes!</b>                      |
| 'a+' (Append & Read) | At the end (EOF)            | Created automatically if absent                                      | Writing always appends; reading requires <code>seek(0)</code> . |

```
# Complete File Lifecycle with seek() and tell()
with open("test.txt", "w+") as f:
    f.write("Python Engineering Master Notes")
    print(f"Pointer Position after write: {f.tell()}") # 32
    f.seek(0) # Reposition pointer to start
    content = f.read()
    print(f"Read Content: '{content}'")
```

## 4.2 OOP Foundations: Classes, Objects, self & Class vs Instance Variables

**Object-Oriented Programming (OOP)** bundles operational behavior (methods) and state (attributes) into single logical units.

- **self Parameter:** An explicit reference to the current instance invoking the method. It allows each instance to access its private attribute space.
- **Instance Variables:** Defined inside methods (typically `__init__`) prefixed with `self.var`. Each object maintains its own isolated copy.
- **Class Variables:** Declared directly within the class body outside any method. Shared universally across all instances of the class.

```
class Student:
    university = "MMU Mullana" # Class Variable (Shared across all instances)

    def __init__(self, roll, name):
        self.roll = roll # Instance Variable (Unique per object)
        self.name = name

s1 = Student(101, "Bhavya")
s2 = Student(102, "Aarav")
Student.university = "MMDU Ambala" # Modifies class state for all instances!
print(s1.university, s2.university) # MMDU Ambala MMDU Ambala
```

## 4.3 Magic (Dunder) Methods & Operator Overloading

Python intercepts built-in operator invocations and maps them directly to underlying **Special (Dunder) Methods**:

| Operator           | Dunder Method             | Function                       | Dunder Method            |
|--------------------|---------------------------|--------------------------------|--------------------------|
| <code>x + y</code> | <code>x.__add__(y)</code> | <code>str(x) / print(x)</code> | <code>x.__str__()</code> |

| Operator | Dunder Method |
|----------|---------------|
| x - y    | x.__sub__(y)  |
| x * y    | x.__mul__(y)  |
| x == y   | x.__eq__(y)   |

| Function | Dunder Method |
|----------|---------------|
| repr(x)  | x.__repr__()  |
| len(x)   | x.__len__()   |
| del x    | x.__del__()   |

```
# Operator Overloading Implementation
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)
    def __str__(self):
        return f"Vector2D({self.x}, {self.y})"

v1 = Vector2D(3, 4)
v2 = Vector2D(1, 2)
print(v1 + v2) # Invokes v1.__add__(v2) -> Output: Vector2D(4, 6)
```

#### 4.4 Inheritance, Method Resolution Order (MRO) & The Diamond Problem

Python supports Single, Multilevel, Multiple, and Hierarchical Inheritance. When multiple parent classes define identical method names (the classic **Diamond Problem**), Python resolves conflicts deterministically via **C3 Linearization (Method Resolution Order - MRO)**.

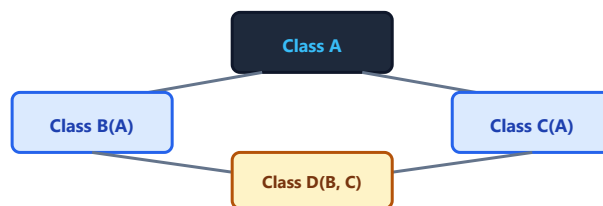


Figure 4.1: The Diamond Inheritance Graph • MRO Resolution Order: D → B → C → A → object

#### 4.5 Polymorphism & Inheritance vs Composition

- **Duck Typing:** Dynamic polymorphism where an object's suitability is determined by the presence of certain methods and properties, rather than its inheritance lineage. ("If it walks like a duck and quacks like a duck, it's a duck.")
- **Composition over Inheritance:** While inheritance models an "Is-A" relationship (e.g., Dog is-a Animal), composition models a "Has-A" relationship (e.g., Car has-an Engine). Composition delivers looser coupling and prevents fragile base class hierarchy bugs.

**Section A: 12 Compulsory 1-Mark Objective & Conceptual Questions****1. Q1: Why is Python designated as a dynamically typed language?**

*Answer:* In Python, variable types are inferred and checked at runtime rather than during compilation. Variables are not declared with types; they merely hold pointers to heap-allocated objects.

**2. Q2: State the output of the expression `2 ** 3 ** 2`.**

*Answer:* 512. The exponentiation operator `**` exhibits **right-to-left associativity**: `3 ** 2 = 9`, then `2 ** 9 = 512`.

**3. Q3: What is the purpose of the `__pycache__` directory?**

*Answer:* It caches compiled platform-independent Python bytecode files (`.pyc`) to accelerate execution startup times on subsequent program invocations.

**4. Q4: Why does `t = (5)` fail to instantiate a tuple object?**

*Answer:* Without a trailing comma, parentheses are evaluated as mathematical grouping operators, creating an integer. The syntax must be `t = (5,)`.

**5. Q5: Distinguish between the `is` operator and the `==` operator.**

*Answer:* `==` evaluates equality of contents/values, whereas `is` evaluates reference identity (verifying whether `id(a) == id(b)`).

**6. Q6: What is the primary operational difference between `break` and `continue`?**

*Answer:* `break` terminates the entire loop immediately, while `continue` aborts only the current iteration and cycles to the next cycle.

**7. Q7: Under what condition does the `else` block attached to a `for` loop execute?**

*Answer:* It executes if and only if the `for` loop terminates naturally upon exhausting its iterable without encountering a `break` statement.

**8. Q8: State the LEGB rule for identifier name resolution.**

*Answer:* Python resolves unqualified variable names through four scopes in order: **Local** → **Enclosing** → **Global** → **Built-in**.

**9. Q9: Why are Python lists categorized as unhashable types?**

*Answer:* Lists are mutable. In-place modification alters their state and would invalidate hash table index buckets, so `__hash__` is set to `None`.

**10. Q10: What is the role of the `self` parameter in Python instance methods?**

*Answer:* `self` provides an explicit reference to the calling instance object, granting the method access to its individual instance variables and methods.

**11. Q11: Explain the purpose of `f.seek(0)` in file handling.**

*Answer:* It resets the internal file cursor pointer to byte offset 0 (the beginning of the file) to enable re-reading the stream from the start.

**12. Q12: Define the term "Duck Typing".**

*Answer:* An application of dynamic polymorphism where the code checks for the presence of specific methods/attributes at runtime rather than explicit class inheritance.

**Section B: 4 Compulsory 2-Mark Analytical Questions****Q13 (Unit 1): Explain why Python does not provide unary `++` or `--` operators. (2 Marks)**

*Solution:* Python integers are immutable objects in heap memory. Permitting `++x` to mutate value in-place would violate fundamental object immutability. Additionally, `++x` is validly parsed by Python's grammar as two consecutive unary positive operators `+(+(x))`, evaluating to `x`. The correct increment idiom is `x += 1`.

**Q14 (Unit 2): What is the Mutable Default Argument trap? Provide code demonstrating the fix. (2 Marks)**

*Solution:* Default argument expressions are evaluated once at function definition time, not at invocation. Using a mutable default (e.g., `def f(x=[])`) causes all invocations to share the identical list across calls. Fix: Use `None` as a sentinel value:

```
def safe_append(val, lst=None):
    if lst is None: lst = []
    lst.append(val)
    return lst
```

**Q15 (Unit 3): Explain the operational mechanics of Shallow Copy vs Deep Copy. (2 Marks)**

*Solution:* A **shallow copy** (`copy.copy()`) constructs a new container object but inserts references into it to the original child objects. A **deep copy** (`copy.deepcopy()`) recursively duplicates all nested objects, creating completely isolated data structures.

**Q16 (Unit 4): Why is the `with open(...)` as `f`: construct preferred over `open()` and `close()`? (2 Marks)**

*Solution:* The `with` statement utilizes Python's context management protocol. It guarantees that the file stream is deterministically closed and OS file descriptors freed via `__exit__()`, even if unhandled exceptions crash the enclosed block.

## Section C: 4 Questions of 4 Marks Each (Selected Representative)

**Q17 (Unit 4 - OOP):** Write a complete Python program illustrating Operator Overloading for complex number addition using the `__add__` and `__str__` dunder methods. (4 Marks)

```
class ComplexNum:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag

    def __add__(self, other):
        return ComplexNum(self.real + other.real, self.imag + other.imag)

    def __str__(self):
        return f"{self.real} + {self.imag}j"

c1 = ComplexNum(3, 4)
c2 = ComplexNum(1, 2)
c3 = c1 + c2 # Invokes c1.__add__(c2)
print("Resultant Complex Sum:", c3) # Output: 4 + 6j
```

## Section D: 4 Questions of 6 Marks Each (University Comprehensive Problem)

**Q18 (Unit 4 - OOP Architecture):** Explain the Diamond Problem in Multiple Inheritance. Illustrate how Python resolves method ambiguity using the C3 Linearization Algorithm and Method Resolution Order (MRO). (6 Marks)

*Solution:* The **Diamond Problem** occurs when class D inherits from classes B and C, both of which inherit from a common base class A. If A defines method `action()`, and both B and C override it, ambiguity arises as to which method D should inherit. Python eliminates this ambiguity deterministically via the **C3 Linearization Algorithm**, which constructs a monotonic precedence list denoted as `__mro__` satisfying three rules: (1) Children precede their parents, (2) Multiple parents maintain their declaration order in the class definition, (3) Monotonicity is strictly preserved.

```
class A:
    def show(self): print("Base A")

class B(A):
    def show(self): print("Derived B")

class C(A):
    def show(self): print("Derived C")

class D(B, C):
    pass

d = D()
d.show() # Outputs: "Derived B" because B precedes C in declaration
print([cls.__name__ for cls in D.__mro__])

>>> ['D', 'B', 'C', 'A', 'object']
```

**Q19 (Unit 4 - File Processing & Text Mining):** Write a complete Python program that reads a text file "sample.txt", counts the total frequency of each unique word (case-insensitive), and prints the top 5 most frequently occurring words in descending order without using external third-party libraries. (6 Marks)

```
def top_word_frequencies(filename, top_n=5):
    word_counts = {}
    with open(filename, 'r') as file:
        for line in file:
            # Clean punctuation and normalize casing
            clean_line = "".join(c.lower() if c.isalnum() or c.isspace() else " " for c in line)
            words = clean_line.split()
            for word in words:
                word_counts[word] = word_counts.get(word, 0) + 1

    # Sort by frequency in descending order
    sorted_words = sorted(word_counts.items(), key=lambda kv: kv[1], reverse=True)
    print(f"Top {top_n} Most Frequent Words:")
    for word, freq in sorted_words[:top_n]:
        print(f"  '{word}': {freq} occurrences")
```

Q20 (Unit 4 - OOP Architecture & Encapsulation): Design a Python class hierarchy modeling a Banking System with a base class Account and derived class SavingsAccount. Implement encapsulation with private attributes, balance validation, deposit/withdraw methods, and custom exception handling for insufficient funds. (6 Marks)

```
class InsufficientFundsError(Exception):
    '''Custom exception raised when withdrawal exceeds available balance.'''
    pass

class Account:
    def __init__(self, acc_num, holder, initial_balance=0.0):
        self.acc_num = acc_num
        self.holder = holder
        self.__balance = float(initial_balance) # Private attribute

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be strictly positive.")
        self.__balance += amount
        return self.__balance

    def withdraw(self, amount):
        if amount > self.__balance:
            raise InsufficientFundsError("Transaction declined: Insufficient funds.")
        self.__balance -= amount
        return self.__balance

    def get_balance(self):
        return self.__balance

class SavingsAccount(Account):
    def __init__(self, acc_num, holder, balance, interest_rate=0.04):
        super().__init__(acc_num, holder, balance)
        self.interest_rate = interest_rate

    def apply_interest(self):
        interest = self.get_balance() * self.interest_rate
        self.deposit(interest)
        return self.get_balance()
```